Asteroids3D

Component-based software engineering project

Student Name: Emil Jørgensen

Student Exam Number: 215661787

GitHub User: Emilioso007

GitHub Repo: https://github.com/Emilioso007/Asteroids3D

0. Abstract

Describe the problem that the report addresses in context of the game domain.

Outline how the developed game addresses the requirement – its key characteristics and fundamental principles (establishing a solution).

Contents

1. Introduction

Asteroids3D is a new take on the classical arcade game, where the player is tasked with navigating a spacecraft through a swarm of asteroids, whilst avoiding enemy bullets along the way. As the name suggests, this game is now in three dimensions, which makes the whole process a lot more challenging. You never know what's behind you.

As the player you control roll, pitch and yaw of your spacecraft, as well as the thrusters and guns. Your task is to crack open asteroids and collect their valuable crystals inside. If an enemy comes close, you may also eliminate them to ease your further travel through space.

2. Requirements

The following requirements are needed to achieve both a fun and extensible game.

Functional requirements for the game:

No.	Title	Description
F01	Player	A controllable player entity.
F02	Enemy	Enemy entities shooting at the player.
F03	Asteroid	Asteroids acting as both obstacles and resources.
F04	Bullet	Bullets being shot by both player and enemy.
F05	Crystal	Crystals as the valuable collectible earning the player points.
F06	Physics	The game handles physics in 3D space.
F07	Rendering	The game renders 3D models with various meshes and materials, including LOD support.
F08	Shader	The game uses shaders to mimic lighting and reflectiveness.

Non-functional requirements for the game:

No.	Title	Description
NF01	Moddable	The game supports loading, updating or removing various components without recompiling the core game.
NF02	Modular	The game uses Java modules to enforce strict encapsulation and prevent tight coupling between components.
NF03	Contracts	The interaction between modules happens through well-defined Service Provider Interfaces, rather than concrete implementations.
NF04	SoC	The game uses a data-oriented ECS architecture, which allows for a great separation of concern between individual systems.
NF05	Microservice	The game provides a microservice for score tally.
NF06	Raylib	The game is using Raylib (Jaylib-ffm) for rendering.
NF07	JPMS	The game is using JPMS and the ServiceLoader to orchestrate everything.
NF08	Spring	The game uses the Spring framework for dependency injection.
NF09	Test	Unit testing is used to ensure the inner workings of modules stay coherent despite changes in architecture.

3. Analysis

The following analysis section describes what the system should do, and what interfaces and entities that could facilitate the requirements stated in section 2. Requirements. This will be documented using various diagrams.

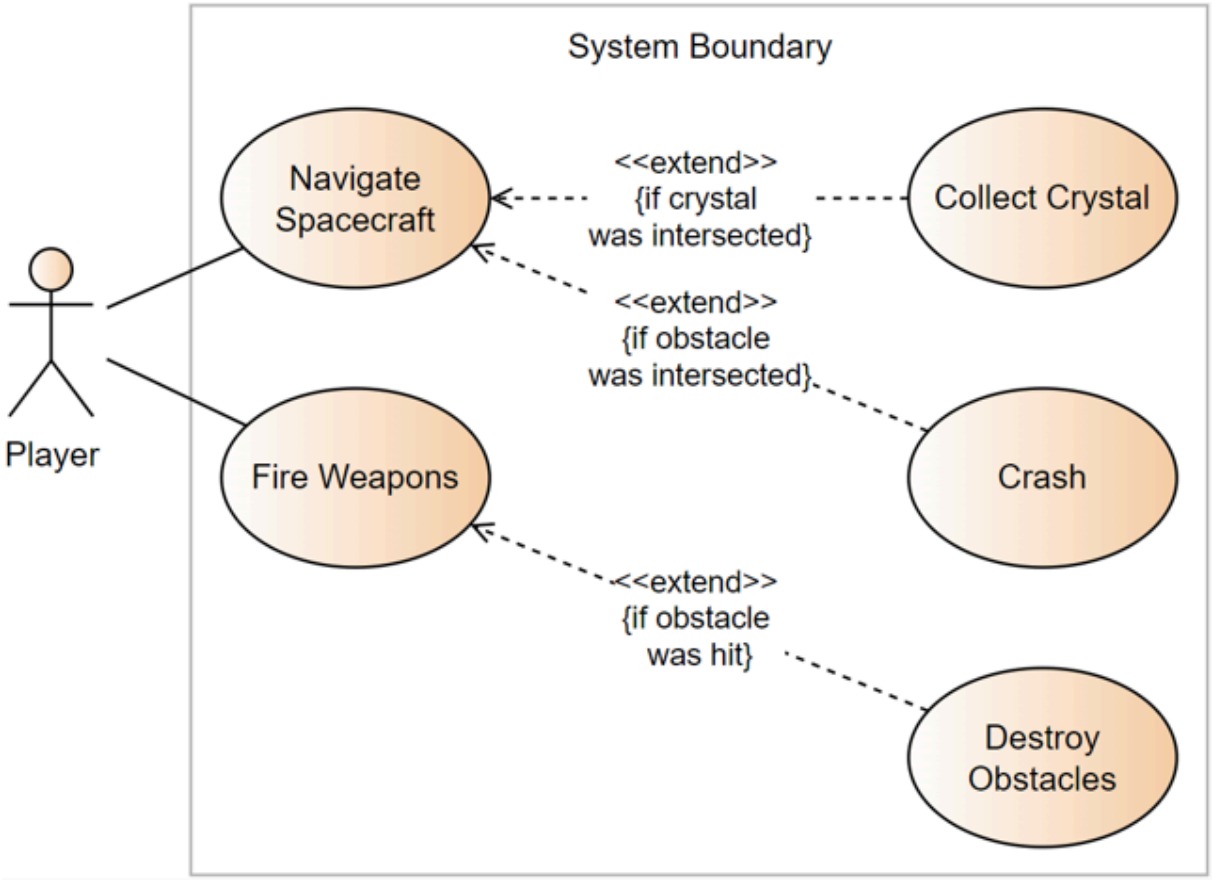

Figure 1: Use Case Diagram that shows what interactions the player can do in the game, as well as what objective each action can lead to.

Based on this diagram (Figure 1) we can see that the system should consist of a player entity that can do various tasks based on user-input.

UC1: When the player navigates the spacecraft, they might intersect a crystal. If so, they collect it and their score is incremented. A high score is the primary goal of the player.

UC2: When the player navigates the spacecraft, they might intersect an obstacle, be it an asteroid, enemy or bullet. This will result in a crash of the spacecraft and game over.

UC3: When the player fires their weapons, they might hit an obstacle, be it an asteroid or enemy. This will destroy said obstacles clearing the path for the player.

This Use Case analysis confirms the proposed entities from the requirements.

- The player is the user-controlled spacecraft.
- The asteroid is an obstacle containing valuable crystals.
- The enemy is an obstacle firing bullets at the player.
- The bullet is a fast-flying entity coming from either the player or the enemies.
- The crystal is the valuable resource.

To link the entities and logic together, some contracts must be provided, namely:

IGamePluginService - Responsible for adding entities to the system.

IEntityProcessorService - Responsible for doing early logic, like movement.

IPostEntityProcessorService - Responsible for doing late logic, like collision detection/resolution.

With these three contracts, it is possible to add entities and perform advanced logic on them.

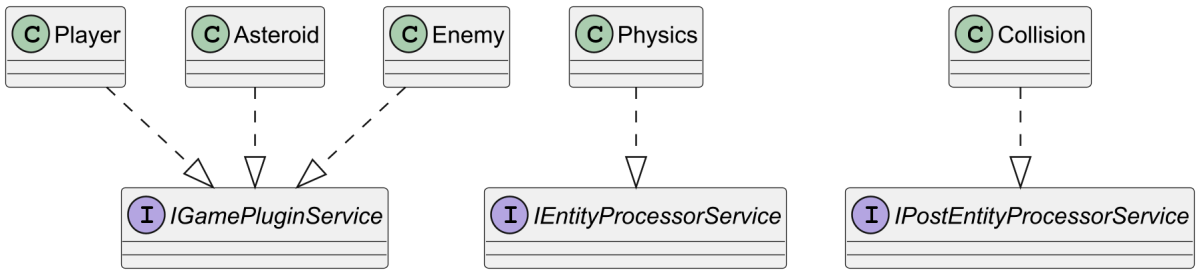

Figure 2: Object Model Diagram showing relation between entities and contracts.

On Figure 2 you can see how these three contracts would be wired up in the system. Bullets and Crystals are both specific edge cases left out for simplicity. The key takeaway is that entities implement the IGamePluginService, telling the system that they want to be added immediately, meanwhile a physics class would implement the IEntityProcessorService, because it needs to calculate movements as the first thing in the frame. A collision class would then implement the IPostEntityProcessorService, since the collision detection and resolution should be done after any movement.

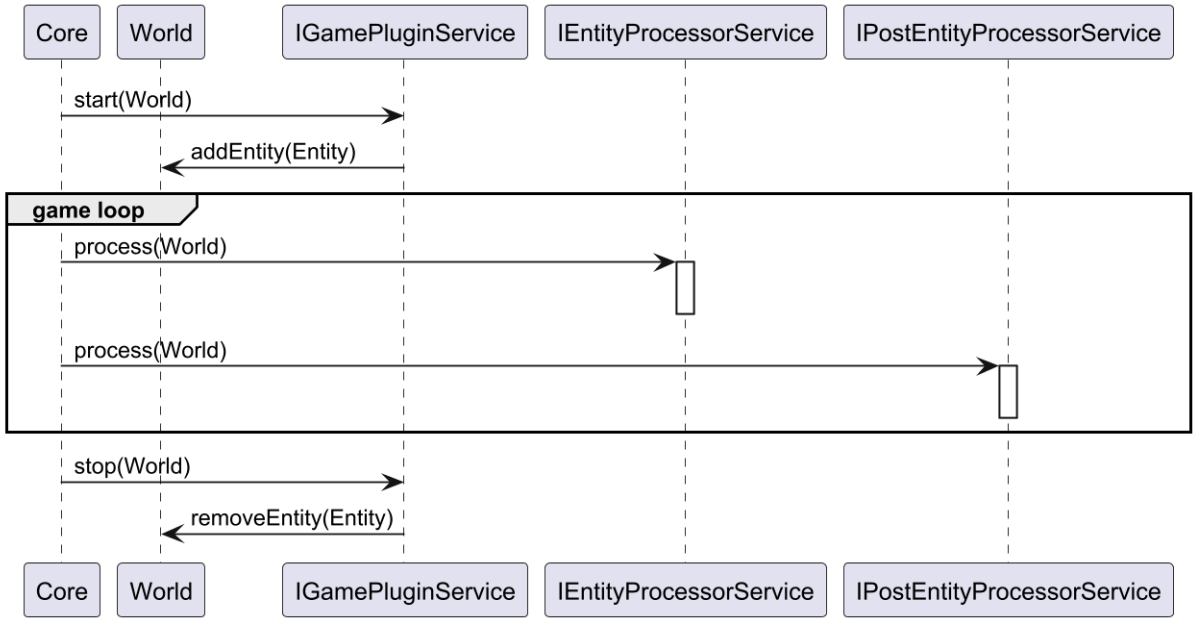

Figure 3: Sequence Diagram showing how Core communicates with the contracts.

On Figure 3 you can see how the three contracts play together in the system. At the very beginning, Core calls the start method on all IGamePluginService implementations with the World as argument. The services would then add their respective entities to the world. Then the main loop start, where the IEntityProcessorService implementations are called first, followed by the IPostEntityProcessorService implementations. Both have a method, process, which takes the world as an argument. Finally, the stop method is invoked, and any remaining entities are removed from the world. The World would contain a reference to all the entities which the processor would access and manipulate as need be.

4. Design

A proper design is crucial in a component-based system. This section will dive into what choices have been made to fulfill all relevant requirements. Complying with requirement NF04, the system is designed in a data-oriented way by utilizing Entity-Component-System architecture traits. Entities serve strictly as containers of various components. There exists zero logic on the individual entities, apart from their creation, which is minimal. Components contain data in all shapes and sizes. This could be everything from just a singular boolean flag, to large meshes and materials.

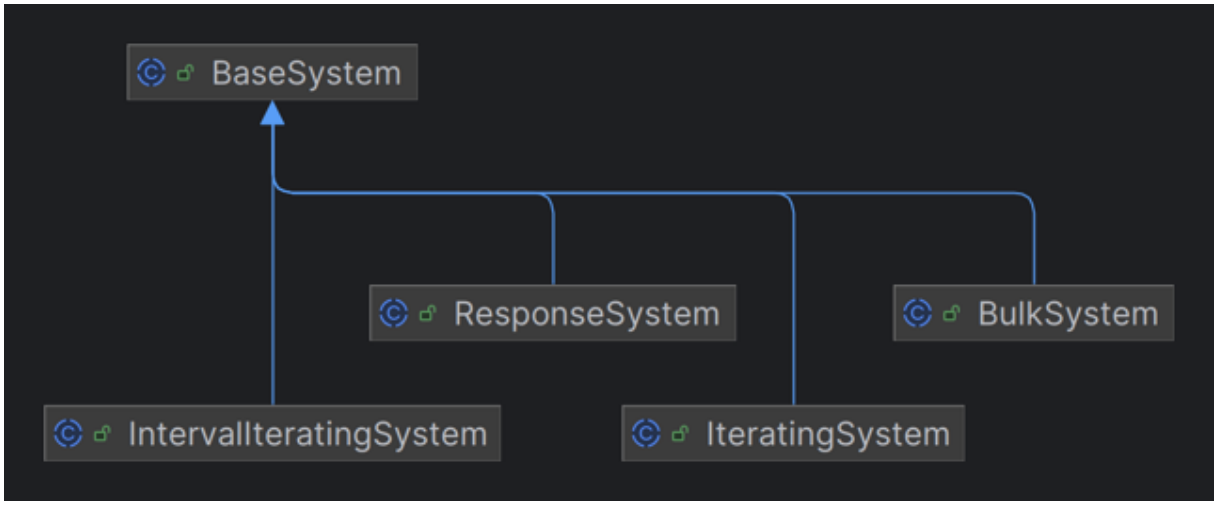

Figure 4: Four sub systems specializing in various patterns.

Systems query the world for entities that contain a collection of components the system is interested in. This is where all the logic is implemented. Different systems have different running characteristics. To accommodate this, four different specialized systems (see Figure 4) have been created, on top of the BaseSystem. The most common is the IteratingSystem. As the name suggests, it iterates through the list of entities matching its signature, one by one. In contrast to the BulkSystem that hands over the entire list of entities all at once, which in some cases is beneficial, for example in a collision detection system where comparison is necessary between entities. The IntervalIteratingSystem is an IteratingSystem on a configurable timer, only firing the update method once per interval. Finally, the ResponseSystem is a way to tell the game that I'm just here to listen to events.

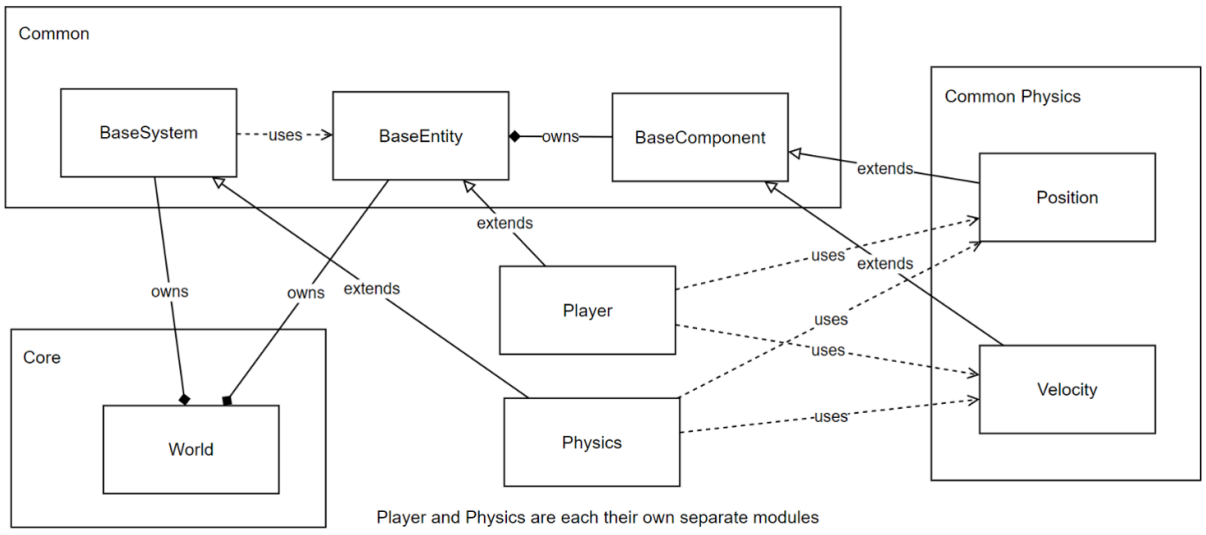

Figure 5: ECS Diagram.

Figure 5 illustrates how the components relate to each other. Notably, there is no direct connection between the Player and Physics modules, despite the Player being affected by physical calculations. This decoupling is achieved through shared contracts. The Player extends the BaseEntity class, which resides in the World. The Physics module is a BaseSystem within that same World. To process entities, the World queries the Physics system through the BaseSystem abstraction to determine what components it expects. Having all the required components serves as the pre-condition for any entity to be processed by that system. The World then filters the entities and passes a list of matching ones to the Physics system. The Position and Velocity components are declared in a common module, that both Physics and Player know about and rely on. This allows the Physics system to manipulate the Player's position without having any dependency on the Player itself. The system's post-condition is fulfilled once it has mutated the relevant component data of the entities. This same principle is applied to all the different systems throughout the game. Another feature of the systems is the priority value. This allows certain systems to run before or after other systems. This is necessary in certain physics calculations where acceleration is applied before velocity. This is also used to ensure that rendering is always the last thing each frame, to make sure that the user is seeing the newest state of the game.

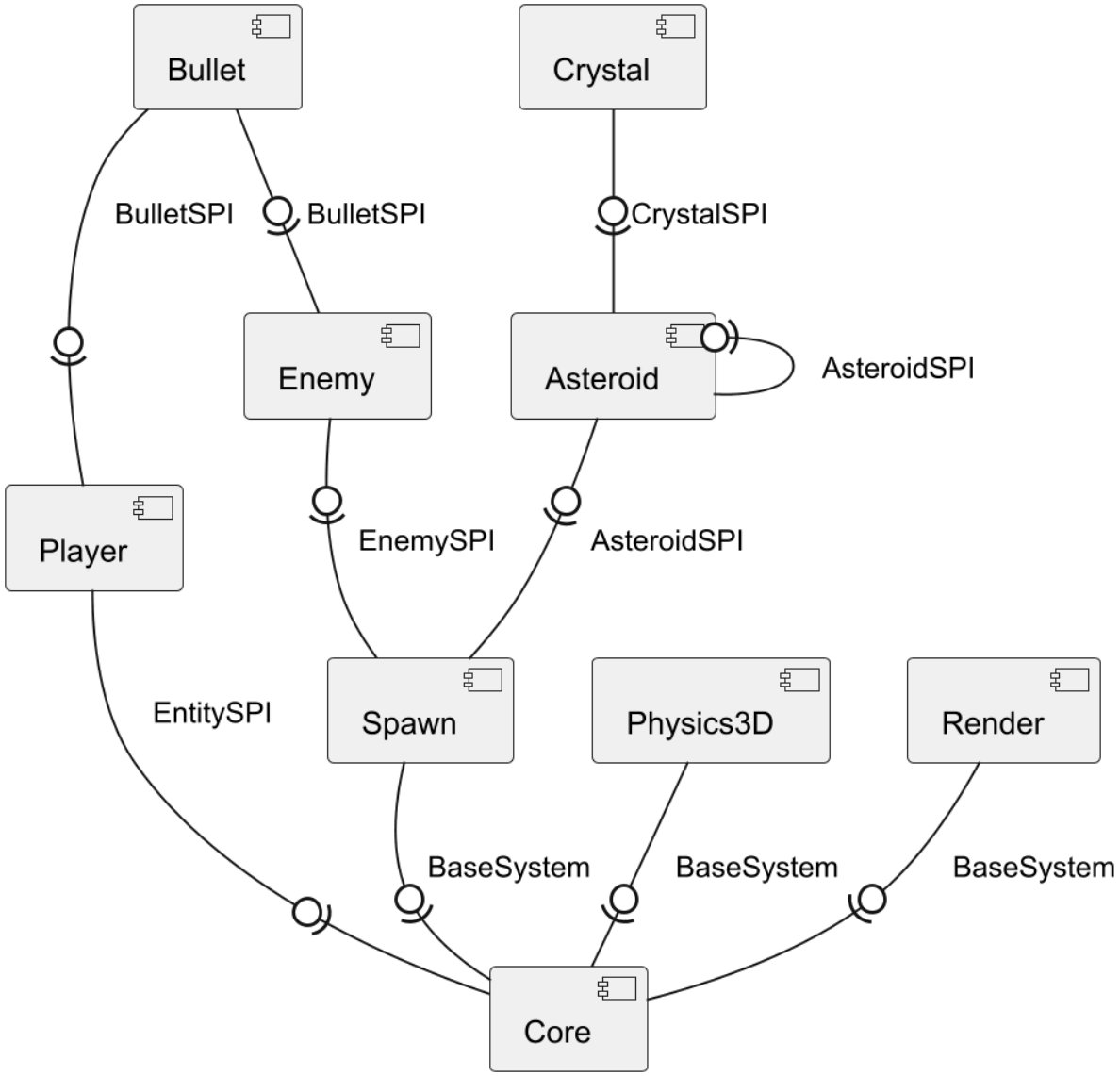

Figure 6: Partial Component Diagram.

Figure 6 shows the modular provides-requires flow of the system. The Player module provides an EntitySPI interface implementation that the Core module uses. Likewise, all the system modules, some of which are shown on the diagram, are providing BaseSystem realizations which the Core module also uses. Enemies and asteroids are controlled by a Spawn system. This requires an Enemy- and AsteroidSPI declared in their own common modules, CommonEnemy and CommonAsteroid respectively. Both Player and Enemy use the BulletSPI interface when shooting. The Asteroid module contains the system spawning crystals on hit, hence the requirement to the CrystalSPI. The discovery between modules is utilizing the Java ServiceLoader pattern, along with internal Spring Dependency Injection. This allows the Core to discover and inject implementations at runtime without any hardcoded dependencies. This satisfies requirements NF01, NF02, NF03, NF07, and NF08. The Scoring Service provides endpoints for incrementing and retrieving the score through a REST-based microservice, satisfying requirement NF05.

5. Implementation

In the following section, the implementation of the system will be documented.

5.1 Modular Encapsulation & Dependencies

To ensure strong encapsulation and reliable dependencies, the Java Platform Module System (JPMS) is used to dictate module accessibility and visibility across the entire system. This is required by NF01, NF02, NF03, and NF07.

```
module Asteroid {
    requires Common;
    requires CommonAsteroid;
    requires CommonCollision;
    requires CommonCrystal;
    requires CommonEnemy;
    requires CommonLifetime;
    requires CommonPhysics3D;
    requires CommonRender;
    requires CommonSpawn;
    requires spring.beans;
    requires spring.context;

    opens io.asteroidsjaylib.asteroid to spring.core, spring.context, spring.beans;

    uses AsteroidSPI;
    uses CrystalSPI;

    provides AsteroidSPI with AsteroidProvider;
    provides BaseSystem with AsteroidCollisionResponseSystem;
}
```

Code Snippet 1: Asteroid module-info file. Imports are excluded.

Code Snippet 1 shows the module-info.java file for the Asteroid module. This module contains the AsteroidProvider, AsteroidEntity, and AsteroidCollisionResponseSystem classes.

The module requires a variety of Common modules. These requires directives tell what other modules must be present for this module to compile and run. This can be seen when compiling the game, where the Common module is the first to get compiled, followed by the other Common modules, and finally the specific implementations.

The opens keyword is used to allow other modules to use reflection within the Asteroid module. Here some sub-packages of the Spring framework are allowed such access. This is required because of the usage of Spring's ApplicationEventPublisher and @EventListener used for event-driven communication throughout the game.

The uses directive signals that this module will act as a consumer, allowing the ServiceLoader to discover external implementations of the specified types. This is here to allow the collision response class to spawn new asteroids and crystals on hit, through the service provider interface implementations.

Finally, the provides … with syntax specifies that this module acts as a service provider. It exposes its internal AsteroidProvider and AsteroidCollisionResponseSystem to any module that uses the AsteroidSPI and BaseSystem types, respectively.

Ultimately, this configuration guarantees that no other module can access the internal implementations of the Asteroid module. By restricting access to everything except the Spring

framework and the explicit SPIs, the system achieves strong encapsulation and reliable dependencies.

```
import io.asteroidsjaylib.common.ecs.BaseSystem;
import io.asteroidsjaylib.common.ecs.EntitySpi;

module Core {
    requires Common;
    requires io.github.electronstudio.jaylib.ffm;
    requires spring.beans;
    requires spring.context;

    opens io.asteroidsjaylib to spring.core, spring.beans, spring.context;

    uses BaseSystem;
    uses EntitySPI;
}
```

Code Snippet 2: Core module-info file.

Code Snippet 2 shows the module-info.java file for the Core module. Here it is worth noting that it only requires the standard common module, and nothing else. It also declares that it uses the BaseSystem class implementations, as well as any EntitySPIs, like the Player. This allows the Core to discover and use the implementing modules through these types, without any dependency on them at compile-time.

5.2 Component Registration & Access

To successfully integrate the decoupled modules at runtime, the system uses a hybrid discovery and injection pattern utilizing both the Java ServiceLoader and the Spring Framework. This satisfies requirements NF01, NF02, NF03, and NF08.

```java
@Configuration
@ComponentScan(basePackages = "io.asteroidsjaylib")
public class AppConfig {

    @Autowired
    private ConfigurableListableBeanFactory beanFactory;

    @Bean
    public List<BaseSystem> baseSystems() {
        List<BaseSystem> systems = new ArrayList<>();
        for (BaseSystem system : ServiceLoader.load(BaseSystem.class)) {

            String beanName = system.getClass().getName();

            beanFactory.registerSingleton(beanName, system);
            beanFactory.autowireBean(system);
            beanFactory.initializeBean(system, beanName);
            systems.add(system);
        }
        return systems;
    }

    @Bean
    public List<EntitySPI> entitySpis() {
        /* Similar logic as above */
    }

}
```

Code Snippet 3: Appconfig class

Code Snippet 3 shows the AppConfig class, which is annotated with @Configuration. This class acts as the bridge between JPMS module discovery and the Spring Inversion of Control (IoC) container. Because the external modules are completely decoupled from the Core, Spring cannot scan their packages directly. To circumvent this, the configuration autowires a ConfigurableListableBeanFactory. Within the bean producer methods, the Java ServiceLoader is used to dynamically discover all provided implementations of BaseSystem and EntitySPI on the module path. Once discovered, these instances are registered with the Spring factory as singletons based on their class names. Spring then autowires and initializes these dynamically loaded classes, bringing them fully under the management of the Spring context.

```java
static void main() {
    AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
    Game game = context.getBean(Game.class);
    game.start();
    context.close();
}

@Autowired
public Game(List<BaseSystem> systems, List<EntitySPI> entitySPIS) {
    this.systems = systems;
    this.entitySPIS = entitySPIS;
}
```

Code Snippet 4: Main method entrypoint and Game constructor.

With the components registered, the application entry point (see Code Snippet 4) initializes the AnnotationConfigApplicationContext using AppConfig.class. It then requests the primary Game bean. The Game class, annotated with @Component, uses constructor injection, explicitly requiring a List and a List as its arguments. Because the AppConfig registered all discovered plugins into the application context, Spring satisfies these dependencies. This now allows the Game to add the systems and entities to the World, without having to use the ServiceLoader directly.

6. Test

7. Discussion

8. Conclusion